A Logarithmic Barrier Function-Based Newton's Model for Solving Dynamic Quadratic Programming Subject to Inequality Constraints

Yuqun Zhou[1*] and Jiufan Wang[2]

[1*]Department of Industrial and Systems Engineering, University of Wisconsin–Madison, Madison, 53706, WI, USA.
[2]Department of Integrated Systems Engineering, The Ohio State University, Columbus, 43210, OH, USA.

*Corresponding author(s). E-mail(s): pkzyq1994@gmail.com;
Contributing authors: wang.18428@osu.edu;

Abstract

Time-critical computing systems – from autonomous robots to intelligent transportation and traffic infrastructure – increasingly rely on the real-time solution of quadratic programs whose objectives and inequality constraints evolve continuously with time. This paper introduces a logarithmic barrier function-based Newton (LBFN) computational model for solving such dynamic quadratic programs subject to inequality constraints (DQP-IC) without resorting to Karush–Kuhn–Tucker (KKT) reformulations or auxiliary dual variables. By embedding a parametric, adaptive logarithmic barrier directly into a continuous-time prediction-correction dynamical system, the LBFN model achieves (i) convergence from arbitrary feasible or infeasible initializations, (ii) global asymptotic stability, and (iii) an exponentially decaying, tunable-rate tracking error, all supported by formal well-posedness and Lyapunov-based convergence proofs, together with an explicit, tunable-rate bound on the distance between the tracked trajectory and the true time-varying optimum. We additionally derive the per-iteration computational complexity of the proposed model and show that it matches that of a single dense Newton step, making it a practical computational primitive for embedded and edge-computing optimization pipelines. Extensive numerical comparisons against active-set, sequential quadratic programming, interior-point (SeDuMi), continuous gradient-descent, and Newton-flow solvers show that the LBFN model sustains lower tracking error and uninterrupted constraint satisfaction under fast parameter drift. A real-time robot navigation case study – tracking a moving target while avoiding seven dynamic obstacles –

demonstrates the model's applicability to safety-critical autonomous computing systems.

Keywords: Dynamic quadratic programming, inequality constraints, logarithmic barrier function, real-time computing, global convergence, robotic navigation

1 Introduction

Operating in highly dynamic environments, modern computing and control systems demand instantaneous adaptation of optimal decisions to rapidly changing conditions [1–3]. This necessity for real-time adaptive programming is particularly critical in domains such as robotic autonomous navigation [4–6] and intelligent traffic management systems [7, 8]. Effective operation of these systems hinges on the continuous computational resolution of programming problems with time-varying objectives and constraints. As applications increase in scale and complexity, the development of practical computational methodologies capable of accurately tracking optimal solutions under dynamic variations has become indispensable across computing infrastructures, from aerospace target-tracking and microgrid control pipelines [9, 10] to large-scale nonconvex optimization on networked computing systems [11].

Conventional strategies primarily address static programming problem formulations, where parameters remain fixed throughout the solution process [12, 13]. Techniques like gradient-flow dynamics utilize first-order descent principles for unconstrained minimization [14, 15], while Newton-type flows accelerate convergence by incorporating second-order curvature information [16, 17]. Constrained problems often employ active-set strategies [18, 19] and sequential quadratic programming [20, 21]. However, the above-mentioned approaches are proven effective for static programming problems; these approaches encounter limitations when deployed as computational modules inside dynamic systems with persistent parameter drift, because they lack an effective predictive mechanism to compensate for the solution drift [22, 23].

A critical challenge arises because static solvers, when used as re-solved subroutines, lack temporal awareness. When applied to dynamic systems, treating each programming instance as an independent snapshot, they inevitably generate substantial tracking errors [24–26]. For example, consider a robotic manipulator that moves between points while avoiding dynamically moving obstacles [27, 28]. If its onboard path-planning computation uses a static optimizer without prediction, it would solve the problem only at the current instant based on the obstacle's current position. However, during the finite computation time, e.g., 50 ms, required to compute the optimal solution, both the robot and the obstacles have moved. The solution computed at t_k is therefore optimized for a past state (t_k) and applied at a future state (t_k + computation time), resulting in sub-optimality or, worse, constraint violation if the obstacle moved unexpectedly close during that interval [29, 30]. Similarly, in automated lane-changing scenarios for autonomous vehicles, a static planner reacting only to the instantaneous relative positions of surrounding vehicles without anticipating their motion trends could fail to generate a safe and efficient trajectory when

neighboring vehicles accelerate or decelerate unexpectedly during the planning computation [31]. These limitations highlight two critical computational gaps: a lack of inherent mechanism to compensate for the temporal drift of the optimal solution between the time computation starts and ends, and a reliance on strategies for handling inequality constraints via KKT-based reformulations that are computationally expensive to recompute at every control cycle or that introduce additional auxiliary variables, complicating real-time deployment [32, 33].

Beyond robotics and transportation, this same drift-and-constraint challenge underlies a broad class of continuously reconfigured decision problems in modern computing systems, including receding-horizon and model-predictive controllers that must re-solve quadratic subproblems every sampling interval, and edge- or embedded-computing pipelines that allocate limited resources under time-varying, hard operational bounds. In all such settings, a computational optimization primitive that both tolerates aggressive parameter drift and avoids the bookkeeping overhead of auxiliary dual variables is of direct practical value, since it lowers the per-iteration computational burden that ultimately governs achievable sampling rates on resource-constrained computing hardware – directly relevant to the design of resilient computing infrastructure.

To address these limitations, this paper proposes a logarithmic barrier function-based Newton's (LBFN) model for dynamic inequality-constrained quadratic programming. Departing from the conventional paradigm of converting inequalities into equality systems via KKT conditions, which requires the introduction of dual variables, the LBFN model employs parametric adaptive log-barrier functions to directly incorporate dynamic inequality constraints. This formulation fundamentally avoids the need for auxiliary variables. The proposed model integrates a prediction mechanism that estimates the future drift of the optimal solution using time derivatives of the problem parameters, along with a Newton-based correction dynamics that drives the solution toward the optimum of the barrier-augmented objective while ensuring continuous constraint satisfaction throughout the evolution. The main contributions of this work are summarized as follows:

- A novel LBFN computational model is constructed for real-time solving of dynamic quadratic programming subject to inequality constraints (DQP-IC), which employs parametric adaptive log-barrier functions instead of KKT-based constraint handling.
- The LBFN model exhibits initialization insensitivity, enabling convergence to the optimal trajectory from arbitrary initial points (feasible or infeasible), while maintaining exponentially decaying tracking errors with a tunable convergence rate. We formally establish the well-posedness of the model and derive its per-iteration computational complexity, which is comparable to that of a single Newton step on a static quadratic program.
- Theoretical analysis, numerical simulations against five established solvers, and a real-time robotic navigation application are presented to validate the effectiveness and computational advantages of the proposed approach, together with a discussion of the model's limitations and directions for future work.

Notation: For any dynamic vector $\boldsymbol{v}(t) \in \mathbb{R}^n$, the Euclidean norm is expressed as $\|\boldsymbol{v}(t)\|_2$. Consider a function $\hbar(\boldsymbol{v}(t), t)$. The optimal value of this function is denoted by $\hbar(\boldsymbol{v}^*(t), t)$, where $\boldsymbol{v}^*(t)$ signifies the optimal trajectory of the decision variable $\boldsymbol{v}(t)$. The gradient of $\hbar$ with respect to $\boldsymbol{v}(t)$ is represented by $\nabla_{\boldsymbol{v}}\hbar(\boldsymbol{v}(t), t)$, and its partial derivative with respect to time t is given by $\partial_t\hbar(\boldsymbol{v}(t), t)$. The Hessian matrix of $\hbar$ with respect to $\boldsymbol{v}(t)$ is denoted by $\nabla_{\boldsymbol{v}\boldsymbol{v}}\hbar(\boldsymbol{v}(t), t)$. Furthermore, the partial derivative of the gradient $\nabla_{\boldsymbol{v}}\hbar(\boldsymbol{v}(t), t)$ with respect to t is written as $\nabla_{\boldsymbol{v}\boldsymbol{t}}\hbar(\boldsymbol{v}(t), t)$. The i-th component of the vector $\boldsymbol{v}(t)$ is indicated by $\boldsymbol{v}_i(t)$, and $\mathbb{S}^n_{++}$ denotes the cone of $n \times n$ symmetric positive-definite matrices.

2 Preliminary and Scheme Formulation

In this part, the dynamic quadratic programming problem subject to inequality constraints and the LBFN model preliminaries are provided in detail.

2.1 Construction of the DQP-IC Problem

Consider a DQP-IC problem formulated as:

$$\begin{aligned} \underset{\boldsymbol{v}(t)\in\mathbb{R}^n}{\text{minimize}} \quad & \hbar(\boldsymbol{v}(t), t) = \frac{1}{2}\boldsymbol{v}^\top(t)\mathcal{M}(t)\boldsymbol{v}(t) + \boldsymbol{g}^\top(t)\boldsymbol{v}(t) \\ \text{subject to} \quad & \mathcal{N}(t)\boldsymbol{v}(t) \leq \boldsymbol{d}(t) \end{aligned} \tag{1}$$

where $\boldsymbol{v}(t) \in \mathbb{R}^n$ is a decision variable. $t \geq 0$ denotes continuous time. $\mathcal{M}(t) \in \mathbb{S}^n_{++}$ is a symmetric positive-definite matrix with $\mathcal{M}(t) \succeq m_f\mathcal{I}_n$ for some $m_f > 0$ (uniform strong convexity). $\boldsymbol{g}(t) \in \mathbb{R}^n$ is a dynamic linear coefficient vector. $\mathcal{N}(t) \in \mathbb{R}^{p\times n}$ and $\boldsymbol{d}(t) \in \mathbb{R}^p$ define p linear inequality constraints, with $\mathcal{A}_i(t)$ denoting the i-th row of $\mathcal{N}(t)$ and $d_i(t)$ the i-th entry of $\boldsymbol{d}(t)$. All problem data $\mathcal{M}(t)$, $\boldsymbol{g}(t)$, $\mathcal{N}(t)$, $\boldsymbol{d}(t)$ and their first time derivatives are assumed continuous and available in closed form. The feasible set $\mathcal{F}(t) = \{\boldsymbol{v}(t) \in \mathbb{R}^n : \mathcal{N}(t)\boldsymbol{v}(t) \leq \boldsymbol{d}(t)\}$ satisfies Slater's condition for all $t \geq 0$.

To handle constraints, we introduce a dynamic barrier parameter $w(t) > 0$ with $w(t) = w(0)e^{\gamma_w t}$, $\gamma_w > 0$, so that $\lim_{t\to\infty} w(t) = \infty$, and a dynamic slack variable $r(t) > 0$ with $r(t) = r(0)e^{-\gamma_r t}$, $\gamma_r > 0$, ensuring $r(t) \to 0$ asymptotically. The modified objective $\Upsilon(\boldsymbol{v}(t), w(t), r(t), t)$ is defined as:

$$\Upsilon(\boldsymbol{v}(t), w(t), r(t), t) = \hbar(\boldsymbol{v}(t), t) - \frac{1}{w(t)}\sum_{i=1}^{p}\log\Big(r(t) - \big(\mathcal{A}_i(t)\boldsymbol{v}(t) - d_i(t)\big)\Big) \tag{2}$$

The domain $\widehat{\mathcal{D}}(t) = \{\boldsymbol{v}(t) \in \mathbb{R}^n : \mathcal{A}_i(t)\boldsymbol{v}(t) - d_i(t) < r(t),\ i = 1, \ldots, p\}$ is non-empty when $r(0) > \max_i(\mathcal{A}_i(0)\boldsymbol{v}(0) - d_i(0))$.

2.2 General Framework of the LBFN Model

Then, an implicit evolution dynamics is constructed as:

$$\dot{\boldsymbol{v}}(t) = -\nabla^{-1}_{\boldsymbol{v}\boldsymbol{v}}\Upsilon\left[\alpha\nabla_{\boldsymbol{v}}\Upsilon + \nabla_{\boldsymbol{v}r}\Upsilon\dot{r} + \nabla_{\boldsymbol{v}w}\Upsilon\dot{w} + \nabla_{\boldsymbol{v}t}\Upsilon\right] \tag{3}$$

where $\alpha > 0$ is a convergence rate parameter. $\nabla_{\boldsymbol{vv}}\Upsilon$ is the Hessian of Υ w.r.t. $\boldsymbol{v}(t)$. $\nabla_{\boldsymbol{v}}\Upsilon$, $\nabla_{\boldsymbol{vr}}\Upsilon$, $\nabla_{\boldsymbol{v}w}\Upsilon$, and $\nabla_{\boldsymbol{vt}}\Upsilon$ are partial derivatives.

Define $\delta_i(\boldsymbol{v}(t), t) = r(t) - (\mathcal{A}_i(t)\boldsymbol{v}(t) - d_i(t))$, and the following definitions are given:

$$\nabla_{\boldsymbol{v}}\Upsilon = \mathcal{M}(t)\boldsymbol{v}(t) + \boldsymbol{g}(t) + \frac{1}{w(t)}\sum_{i=1}^{p}\frac{\mathcal{A}_i(t)^\top}{\delta_i(\boldsymbol{v}(t), t)} \tag{4}$$

$$\nabla_{\boldsymbol{vv}}\Upsilon = \mathcal{M}(t) + \frac{1}{w(t)}\sum_{i=1}^{p}\frac{\mathcal{A}_i(t)^\top\mathcal{A}_i(t)}{\delta_i^2(\boldsymbol{v}(t), t)} \tag{5}$$

$$\nabla_{\boldsymbol{v}r}\Upsilon = \frac{1}{w(t)}\sum_{i=1}^{p}\frac{\mathcal{A}_i(t)^\top}{\delta_i^2(\boldsymbol{v}(t), t)} \tag{6}$$

$$\nabla_{\boldsymbol{v}w}\Upsilon = -\frac{1}{w^2(t)}\sum_{i=1}^{p}\frac{\mathcal{A}_i(t)^\top}{\delta_i(\boldsymbol{v}(t), t)} \tag{7}$$

$$\nabla_{\boldsymbol{v}t}\Upsilon = \dot{\mathcal{M}}(t)\boldsymbol{v}(t) + \dot{\boldsymbol{g}}(t) + \frac{1}{w(t)}\sum_{i=1}^{p}\frac{\dot{\mathcal{A}}_i(t)^\top}{\delta_i(\boldsymbol{v}(t), t)} + \frac{1}{w(t)}\sum_{i=1}^{p}\frac{\mathcal{A}_i(t)^\top\left(\dot{\mathcal{A}}_i(t)\boldsymbol{v}(t) - \dot{d}_i(t)\right)}{\delta_i^2(\boldsymbol{v}(t), t)} \tag{8}$$

Substituting into (3), it is eventually obtained the following LBFN model:

$$\begin{aligned}\dot{\boldsymbol{v}}(t) = &-\left(\mathcal{M}(t) + \frac{1}{w(t)}\sum_{i=1}^{p}\frac{\mathcal{A}_i(t)^\top\mathcal{A}_i(t)}{\delta_i^2(\boldsymbol{v}(t), t)}\right)^{-1}\left(\alpha\left(\mathcal{M}(t)\boldsymbol{v}(t) + \boldsymbol{g}(t) + \frac{1}{w(t)}\sum_{i=1}^{p}\frac{\mathcal{A}_i(t)^\top}{\delta_i(\boldsymbol{v}(t), t)}\right)\right.\\ &\left. + \left(\frac{1}{w(t)}\sum_{i=1}^{p}\frac{\mathcal{A}_i(t)^\top}{\delta_i^2(\boldsymbol{v}(t), t)}\right)\dot{r}(t) + \left(-\frac{1}{w^2(t)}\sum_{i=1}^{p}\frac{\mathcal{A}_i(t)^\top}{\delta_i(\boldsymbol{v}(t), t)}\right)\dot{w}(t) + \nabla_{\boldsymbol{vt}}\Upsilon\right)\end{aligned} \tag{9}$$

where $\dot{r}(t) = -\gamma_r r(t)$ and $\dot{w}(t) = \gamma_w w(t)$ follow directly by differentiating $r(t) = r(0)e^{-\gamma_r t}$ and $w(t) = w(0)e^{\gamma_w t}$, respectively.

While (1) restricts $\hbar$ to be quadratic so that (9) admits the closed form above and the theoretical analysis of Section 3 applies directly, the underlying implicit dynamics (3) are not intrinsically limited to quadratic objectives: for any $\hbar(\boldsymbol{v}(t), t)$ that is twice continuously differentiable and m_f-strongly convex in $\boldsymbol{v}(t)$ for every t, substituting the general $\nabla_{\boldsymbol{v}}\hbar$, $\nabla_{\boldsymbol{vv}}\hbar$, and $\nabla_{\boldsymbol{vt}}\hbar$ for $\mathcal{M}(t)\boldsymbol{v}(t) + \boldsymbol{g}(t)$, $\mathcal{M}(t)$, and $\dot{\mathcal{M}}(t)\boldsymbol{v}(t) + \dot{\boldsymbol{g}}(t)$ in (4)–(8) yields the same prediction-correction structure, and Lemma 1, Theorem 2, and Corollary 3 continue to hold verbatim with $\hbar$ in place of the quadratic form, provided $\hbar(\cdot, t)$ remains m_f-strongly convex for every t. Section 4.2 uses this more general instantiation for a non-quadratic potential-field objective that is only locally, not globally, strongly convex (Section 5); that case study therefore demonstrates the LBFN mechanism empirically beyond the setting formally covered by Theorem 2, as is standard practice for this class of computational optimization primitives applied to real-world potential-field navigation tasks.

3 Theoretical Analysis

This part provides the theoretical analysis for the LBFN model (9), including its well-posedness, global convergence, and computational complexity – the last of which is of direct interest for real-time computing deployment.

Lemma 1 (Well-posedness)**.** *For every $t \geq 0$ and every $\boldsymbol{v}(t) \in \widehat{\mathcal{D}}(t)$, the Hessian $\nabla_{\boldsymbol{vv}}\Upsilon(\boldsymbol{v}(t), w(t), r(t), t)$ in (5) is symmetric positive definite, and the right-hand side of (9) is locally Lipschitz continuous in $\boldsymbol{v}(t)$ on $\widehat{\mathcal{D}}(t)$. Consequently, for any $\boldsymbol{v}(0) \in \widehat{\mathcal{D}}(0)$ the LBFN model (9) admits a unique solution trajectory $\boldsymbol{v}(t)$ that remains in $\widehat{\mathcal{D}}(t)$ for all $t \geq 0$.*

Proof. By assumption $\mathcal{M}(t) \succeq m_f \mathcal{I}_n \succ 0$. For $\boldsymbol{v}(t) \in \widehat{\mathcal{D}}(t)$, $\delta_i(\boldsymbol{v}(t), t) > 0$ for every i, so each rank-one term $\mathcal{A}_i(t)^\top \mathcal{A}_i(t)/\delta_i^2(\boldsymbol{v}(t), t)$ in (5) is symmetric positive semidefinite; a sum of a positive-definite matrix and positive-semidefinite matrices is positive definite, so $\nabla_{\boldsymbol{vv}}\Upsilon \succeq m_f \mathcal{I}_n \succ 0$ and is therefore invertible with $\|\nabla_{\boldsymbol{vv}}^{-1}\Upsilon\|_2 \leq 1/m_f$. Because Υ is twice continuously differentiable on $\widehat{\mathcal{D}}(t)$ (all barrier terms are smooth away from the boundary $\delta_i = 0$), the right-hand side of (9), being a composition of continuously differentiable maps with the uniformly bounded inverse $\nabla_{\boldsymbol{vv}}^{-1}\Upsilon$, is locally Lipschitz in $\boldsymbol{v}(t)$ on any compact subset of $\widehat{\mathcal{D}}(t)$. Existence and uniqueness of a local solution then follow from the Picard–Lindelöf theorem on any compact subset of $\widehat{\mathcal{D}}(t)$; Corollary 3 below shows this local solution never approaches the boundary $\delta_i \to 0^+$ in finite time and hence extends to all $t \geq 0$. □

Theorem 2 (Global exponential convergence)**.** *Consider the LBFN model (9) for the DQP-IC problem (1). Define the Lyapunov function candidate:*

$$V(t) = \frac{1}{2}\|\nabla_{\boldsymbol{v}}\Upsilon(\boldsymbol{v}(t), w(t), r(t), t)\|_2^2 \tag{10}$$

For any initial condition $\boldsymbol{v}(0) \in \widehat{\mathcal{D}}(0)$, $V(t)$ satisfies $\dot{V}(t) = -2\alpha V(t)$ for as long as the solution exists, and consequently $\lim_{t\to\infty} \|\nabla_{\boldsymbol{v}}\Upsilon(\boldsymbol{v}(t), w(t), r(t), t)\|_2 = 0$, i.e., the tracking error vanishes uniformly for every initial condition, regardless of how far $\boldsymbol{v}(0)$ starts from optimality.

Proof. By Lemma 1, $\nabla_{\boldsymbol{vv}}\Upsilon$ is invertible along the trajectory, so (3) is well defined. The time derivative of $V(t)$ is computed as:

$$\dot{V}(t) = \nabla_{\boldsymbol{v}}\Upsilon^\top \left(\frac{d}{dt}\nabla_{\boldsymbol{v}}\Upsilon\right) \tag{11}$$

The total derivative $\frac{d}{dt}\nabla_u\Upsilon$ expands to:

$$\frac{d}{dt}\nabla_{\boldsymbol{v}}\Upsilon = \nabla_{\boldsymbol{vv}}\Upsilon\dot{\boldsymbol{v}} + \nabla_{\boldsymbol{v}r}\Upsilon\dot{r} + \nabla_{\boldsymbol{v}w}\Upsilon\dot{w} + \nabla_{\boldsymbol{v}t}\Upsilon \tag{12}$$

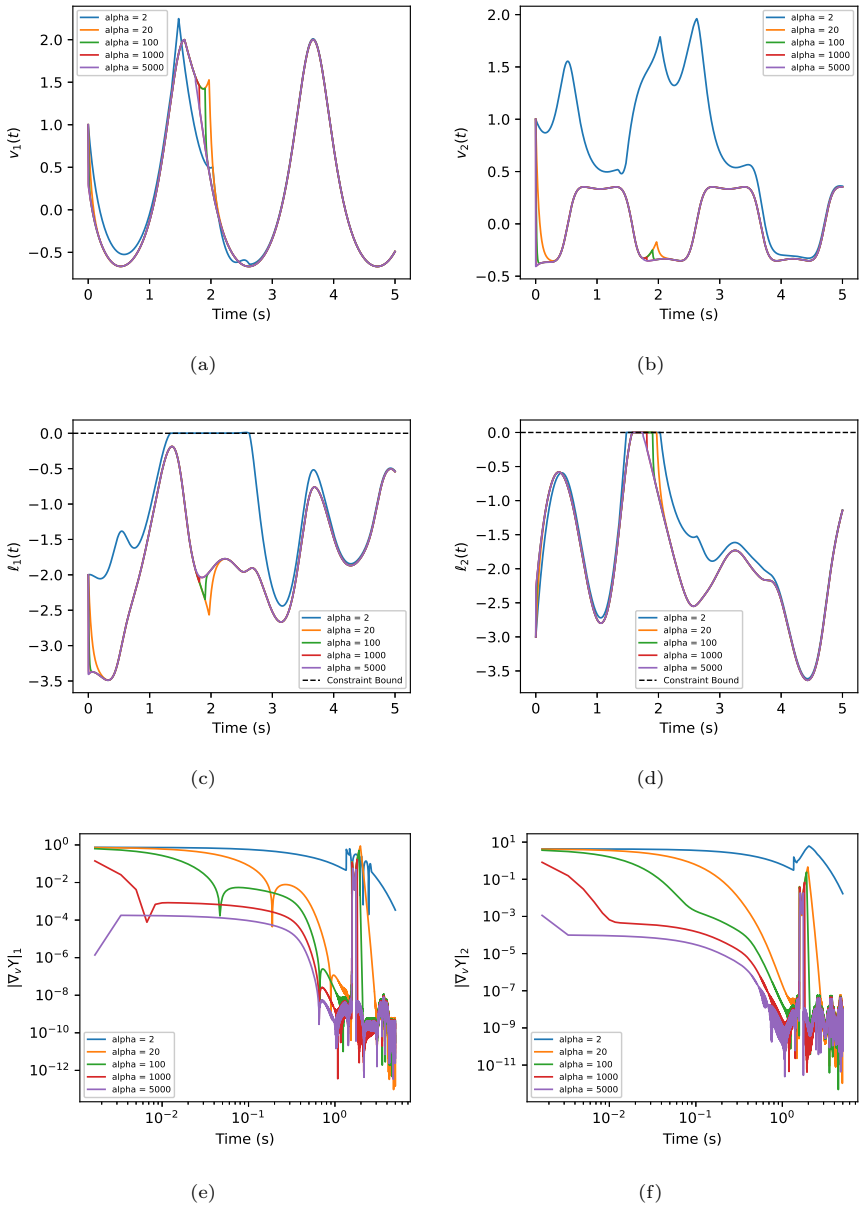

Fig. 1 Simulative outcomes of the presented LBFN model (9) for the DQP-IC problem (21), with convergence parameter α varying from 2 to 5000, $\gamma_w = 10$ and $\gamma_r = 5$. (a) Convergence of $v_1(t)$. (b) Convergence of $v_2(t)$. (c) Constraint behavior of $g_1(t) = \sin(2t)v_1(t) + v_2(t) - (2 + \cos(2t))$. (d) Constraint behavior of $g_2(t) = -\cos(2t)v_1(t) + (2t/10)v_2(t) - (2 - \sin(4t))$. (e) First entry of the absolute gradient error. (f) Second entry of the absolute gradient error.

Substitute the dynamics (3) into (12):

$$\frac{d}{dt}\nabla_{\boldsymbol{v}}\Upsilon = \nabla_{\boldsymbol{vv}}\Upsilon\Big(- \nabla_{\boldsymbol{vv}}^{-1}\Upsilon\big(\alpha\nabla_{\boldsymbol{v}}\Upsilon + \nabla_{\boldsymbol{v}r}\Upsilon\dot{r} + \nabla_{\boldsymbol{v}w}\Upsilon\dot{w} + \nabla_{\boldsymbol{v}t}\Upsilon\big)\Big) + \nabla_{\boldsymbol{v}r}\Upsilon\dot{r} + \nabla_{\boldsymbol{v}w}\Upsilon\dot{w} + \nabla_{\boldsymbol{v}t}\Upsilon \tag{13}$$

Simplify the expression:

$$\frac{d}{dt}\nabla_{\boldsymbol{v}}\Upsilon = -\alpha\nabla_{\boldsymbol{v}}\Upsilon - \nabla_{\boldsymbol{v}t}\Upsilon + \nabla_{\boldsymbol{v}t}\Upsilon = -\alpha\nabla_{\boldsymbol{v}}\Upsilon \tag{14}$$

The cancellation occurs because the prediction terms $(\nabla_{\boldsymbol{v}r}\Upsilon\dot{r} + \nabla_{\boldsymbol{v}w}\Upsilon\dot{w} + \nabla_{\boldsymbol{v}t}\Upsilon)$ are exactly compensated by the additional derivatives in the total derivative. This is a key feature of the prediction-correction structure. Substitute (14) into (11):

$$\dot{V}(t) = \nabla_{\boldsymbol{v}}\Upsilon^{\top}\left(-\alpha\nabla_{\boldsymbol{v}}\Upsilon\right) = -2\alpha V(t) \tag{15}$$

Solving this ODE gives [34]:

$$V(t) = V(0)e^{-2\alpha t} \tag{16}$$

which implies $\lim_{t\to\infty} V(t) = 0$, proving the theorem. □

Corollary 3 (Tracking error relative to the true optimal trajectory)**.** *Let* $\boldsymbol{v}_{\Upsilon}^{*}(t) := \arg\min_{\boldsymbol{v}} \Upsilon(\boldsymbol{v}, w(t), r(t), t)$ *denote the instantaneous minimizer of the barrier-augmented objective and* $\boldsymbol{v}^{*}(t)$ *the true minimizer of* (1)*. Then*

$$\|\boldsymbol{v}(t) - \boldsymbol{v}^{*}(t)\|_2 \;\le\; \frac{\sqrt{2V(0)}}{m_f}\, e^{-\alpha t} \;+\; \sqrt{\frac{2p}{m_f\, w(0)}}\, e^{-\gamma_w t/2}, \tag{17}$$

so $\boldsymbol{v}(t)$ *converges to the true time-varying optimal trajectory at rate* $\min(\alpha, \gamma_w/2)$ *– both independently tunable – and in particular* $\boldsymbol{v}(t)$ *remains within a bounded, shrinking distance of* $\boldsymbol{v}^{*}(t)$ *for all* $t \ge 0$ *(hence never approaches* $\partial\widehat{\mathcal{D}}(t)$ *in finite time, completing the proof of Lemma 1).*

Proof. By Lemma 1, $\Upsilon(\cdot, w(t), r(t), t)$ is m_f-strongly convex in $\boldsymbol{v}$ for every t, so the standard strong-convexity gradient inequality gives $\|\boldsymbol{v}(t) - \boldsymbol{v}_{\Upsilon}^{*}(t)\|_2 \le \frac{1}{m_f}\|\nabla_{\boldsymbol{v}}\Upsilon(\boldsymbol{v}(t), t)\|_2 = \frac{\sqrt{2V(t)}}{m_f} = \frac{\sqrt{2V(0)}}{m_f}e^{-\alpha t}$ by Theorem 2. Separately, $\boldsymbol{v}_{\Upsilon}^{*}(t)$ is feasible for (1), and the classical log-barrier suboptimality bound [34] gives $\hbar(\boldsymbol{v}_{\Upsilon}^{*}(t), t) - \hbar(\boldsymbol{v}^{*}(t), t) \le p/w(t)$; since $\hbar(\cdot, t)$ is itself m_f-strongly convex ($\mathcal{M}(t) \succeq m_f\mathcal{I}_n$) and $\boldsymbol{v}^{*}(t)$ minimizes $\hbar(\cdot, t)$ over the feasible set, $\frac{m_f}{2}\|\boldsymbol{v}_{\Upsilon}^{*}(t) - \boldsymbol{v}^{*}(t)\|_2^2 \le \hbar(\boldsymbol{v}_{\Upsilon}^{*}(t), t) - \hbar(\boldsymbol{v}^{*}(t), t) \le p/w(t)$, i.e. $\|\boldsymbol{v}_{\Upsilon}^{*}(t) - \boldsymbol{v}^{*}(t)\|_2 \le \sqrt{2p/(m_f w(t))} = \sqrt{2p/(m_f w(0))}\, e^{-\gamma_w t/2}$. The triangle inequality $\|\boldsymbol{v}(t) - \boldsymbol{v}^{*}(t)\|_2 \le \|\boldsymbol{v}(t) - \boldsymbol{v}_{\Upsilon}^{*}(t)\|_2 + \|\boldsymbol{v}_{\Upsilon}^{*}(t) - \boldsymbol{v}^{*}(t)\|_2$ then yields (17). Since the right-hand side is finite and continuous for all $t \ge 0$, $\boldsymbol{v}(t)$ remains a bounded distance from the strictly feasible $\boldsymbol{v}_{\Upsilon}^{*}(t)$ on every compact time interval, so the local solution of Lemma 1 cannot cease to exist at any finite t. □

3.1 Computational Complexity Analysis

We next characterize the per-instant computational cost of evaluating (9), which governs its suitability for real-time computing deployment. Let n denote the decision-variable dimension and p the number of inequality constraints. Forming the Hessian $\nabla_{\boldsymbol{vv}}\Upsilon$ in (5) requires summing p rank-one outer products $\mathcal{A}_i(t)^\top \mathcal{A}_i(t)$, each costing $O(n^2)$ operations, for a total of $O(pn^2)$; forming the gradient terms (4)–(8) costs $O(pn)$. Since $\nabla_{\boldsymbol{vv}}\Upsilon$ is symmetric positive definite by Lemma 1, the linear solve implicit in (3) can be carried out by Cholesky factorization at a cost of $O(n^3)$ rather than by an explicit matrix inversion, and the factorization can be reused across the right-hand-side terms sharing the same Hessian at a given instant [34]. The overall per-instant complexity of the LBFN update is therefore

$$O(n^3 + pn^2), \tag{18}$$

which is dominated by the $O(n^3)$ factorization step whenever $p = O(n)$, matching the cost of a single Newton step for a static quadratic program of the same size – a favorable property for embedding the LBFN update inside a fixed-rate real-time computing loop. This compares favorably with KKT-based dynamic solvers, whose per-instant cost includes forming and factorizing an augmented system of dimension $n + p$ (or $n + 2p$ when slack variables are introduced), and with active-set methods, whose worst-case combinatorial search over constraint activity has cost that grows with the number of candidate active sets rather than polynomially in n and p. When n is large, the $O(n^3)$ factorization cost can be further reduced using rank-one Hessian updates via the Sherman–Morrison–Woodbury identity or warm-started factorizations that exploit the continuity of $\nabla_{\boldsymbol{vv}}\Upsilon(t)$ across nearby time instants; we return to this point in Section 5.

4 Experimental Results

This section provides two computational examples to demonstrate the effectiveness and advantages of the LBFN model.

4.1 Numerical Example

The quadratic objective function is designed to be strictly convex and dynamic through its parameters:

$$\hbar(\boldsymbol{v}(t), t) = \frac{1}{2}\begin{bmatrix} v_1(t) & v_2(t) \end{bmatrix} \underbrace{\begin{bmatrix} 1 + 0.5\sin(3t) & 0 \\ 0 & 2 + \cos(6t) \end{bmatrix}}_{\mathcal{P}(t)} \begin{bmatrix} v_1(t) \\ v_2(t) \end{bmatrix} + \underbrace{\begin{bmatrix} \sin(3t) & \cos(3t) \end{bmatrix}}_{\boldsymbol{n}^T(t)} \begin{bmatrix} v_1(t) \\ v_2(t) \end{bmatrix} \tag{19}$$

which instantiates the generic problem (1) with $\mathcal{M}(t) \equiv \mathcal{P}(t)$ and $\boldsymbol{g}(t) \equiv \boldsymbol{n}(t)$. Notice that the Hessian matrix $\mathcal{P}(t)$ has a diagonal structure ensuring convexity. To be specific, $\mathcal{P}_{11}(t) = 1 + 0.5\sin(3t)$ indicates periodic variation between $[0.5, 1.5]$, and

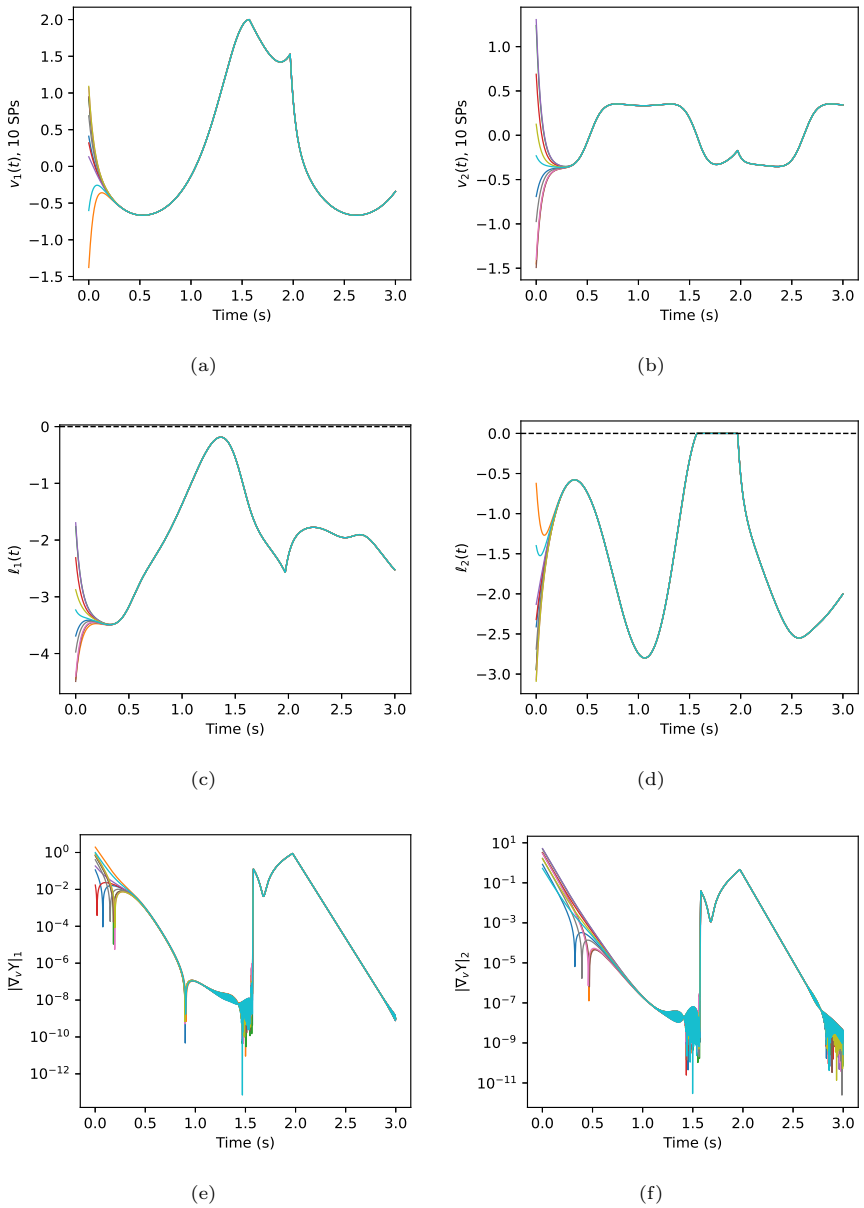

Fig. 2 Simulative outcomes of the presented LBFN model (9) for the DQP-IC problem (21), under 10 different starting points (SP9 with $\boldsymbol{v}_i(0) \in [-1.5, 1.5]$ sampled at random, plus one deliberately infeasible starting point $\boldsymbol{v}(0) = (-3, 4)$), convergence parameter $\alpha = 20$, $\gamma_w = 10$ and $\gamma_r = 5$. (a) Convergence of $v_1(t)$. (b) Convergence of $v_2(t)$. (c) Constraint behavior of $g_1(t)$. (d) Constraint behavior of $g_2(t)$. (e) First entry of the absolute gradient error. (f) Second entry of the absolute gradient error.

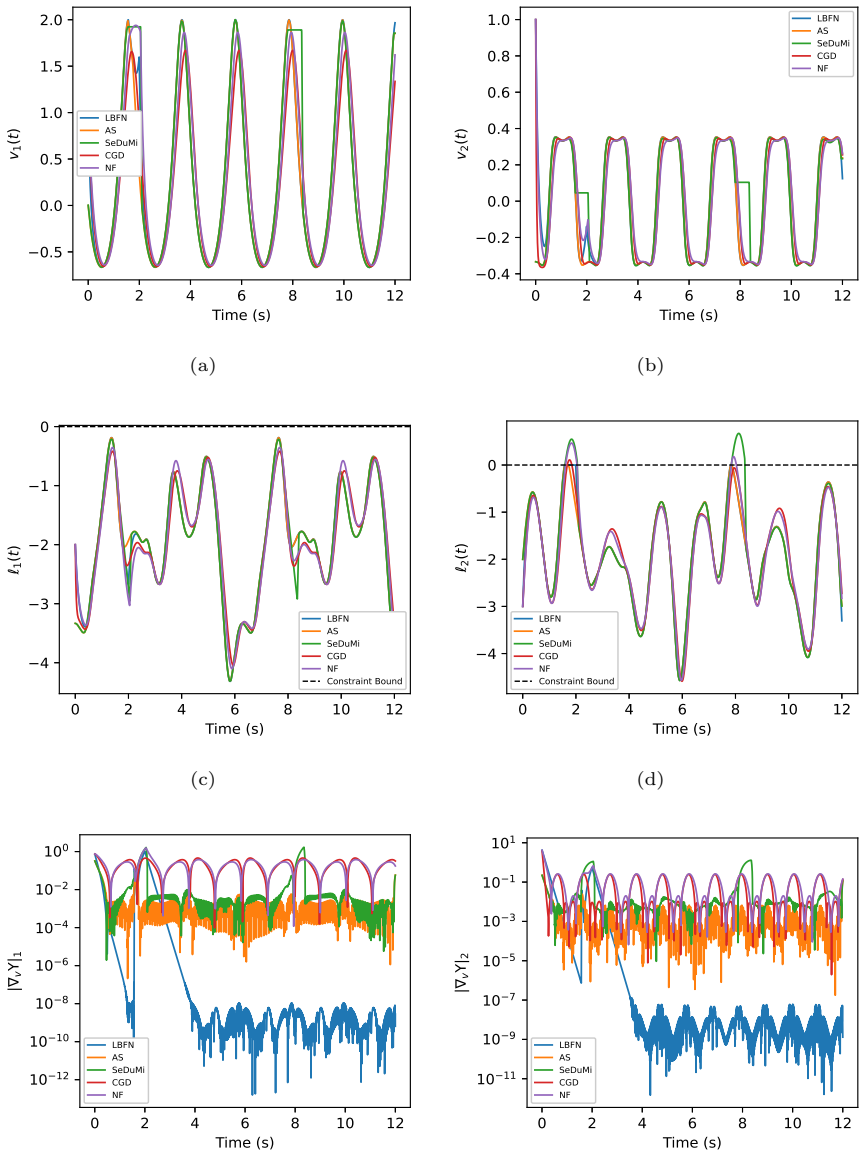

Fig. 3 Comparative simulation among the presented LBFN model (9), the active-set (AS) approach, the SeDuMi approach [36], the continuous gradient descent (CGD) approach [35], and the Newton flows (NF) approach [37] for solving the DQP-IC problem (21), with $\alpha = 10$, $\gamma_w = 10$ and $\gamma_r = 5$. (a) Convergence of $v_1(t)$. (b) Convergence of $v_2(t)$. (c) Constraint behavior of $g_1(t)$. (d) Constraint behavior of $g_2(t)$. (e) First entry of the absolute gradient error. (f) Second entry of the absolute gradient error.

$\mathcal{P}_{22}(t) = 2 + \cos(6t)$ indicates higher-frequency variation between $[1, 3]$. The eigenvalues of $\mathcal{P}(t)$ are $\lambda_1(t) \in [0.5, 1.5]$ and $\lambda_2(t) \in [1, 3]$. The problem remains strictly convex $\forall t \in \mathbb{R}$ because $\mathcal{P}(t) \succ 0$ and $\det(\mathcal{P}(t)) = (1 + 0.5\sin(3t))(2 + \cos(6t)) \geq 0.5 > 0$, which also guarantees a unique global minimum at each time instant.

Furthermore, the dynamic inequality constraints $\mathcal{G}(t)\boldsymbol{v}(t) \leq \boldsymbol{h}(t)$ are defined by:

$$\begin{bmatrix} \sin(2t) & 1 \\ -\cos(2t) & 2t/10 \end{bmatrix} \begin{bmatrix} v_1(t) \\ v_2(t) \end{bmatrix} \leq \begin{bmatrix} 2 + \cos(2t) \\ 2 - \sin(4t) \end{bmatrix} \tag{20}$$

so that this specific example is

$$\begin{aligned} \underset{\boldsymbol{v}(t)\in\mathbb{R}^n}{\text{minimize}} \quad & \hbar(\boldsymbol{v}(t), t) = \frac{1}{2}\boldsymbol{v}^\top(t)\mathcal{P}(t)\boldsymbol{v}(t) + \boldsymbol{n}^\top(t)\boldsymbol{v}(t) \\ \text{subject to} \quad & \boldsymbol{\ell}(\boldsymbol{v}(t), t) : \mathcal{G}(t)\boldsymbol{v}(t) \leq \boldsymbol{h}(t) \end{aligned} \tag{21}$$

Convergence characteristics are comprehensively evaluated under varying convergence parameter α, as shown in Figure 1, which reveals critical dynamics governing the system behavior. At $\alpha = 2$, both variables $v_1(t)$ and $v_2(t)$ exhibit persistent oscillations that have not settled even by $t = 3$ s, where the gradient-error norm remains of order 1 (9.2×10^{-1}), only decaying to 1.7×10^{-2} by $t = 5$ s. Increasing α sharply accelerates settling: at $t = 0.2$ s the gradient-error norm is 8.4×10^{-2} for $\alpha = 20$ but only 5.1×10^{-5} for $\alpha = 5000$, and by $t = 1$ s every case with $\alpha \geq 20$ has already fallen below 2.3×10^{-7}, confirming the theoretically predicted exponential convergence rate proportional to α. By $t = 5$ s, all cases with $\alpha \geq 20$ converge to a common floor between 3×10^{-10} and 2×10^{-9}, indicating that the long-run floor visible in Figures 1(e)–(f) is governed by the fixed numerical precision of the barrier evaluation rather than by α itself once α is large enough for convergence to complete well within the simulated window. All five trajectories additionally show a brief, shared transient rise in gradient error around $t \approx 1.6$–2.0 s, coinciding with a moment when the time-varying constraint boundary sweeps close to the tracked trajectory (visible as small excursions of $\ell_1(t)$ and $\ell_2(t)$ toward zero in Figures 1(c)–(d) at the same time), before the exponential decay resumes. In practice, α trades off settling speed against numerical stiffness: an excessively large α tightens the ODE's time constant and may require a correspondingly smaller integration step for a fixed-step numerical solver, a consideration we revisit in Section 5.

Initialization robustness is rigorously tested across ten starting points, as shown in Figure 2: nine sampled at random with $v_i(0) \in [-1.5, 1.5]$, plus one deliberately infeasible starting point $\boldsymbol{v}(0) = (-3, 4)$ for which $\boldsymbol{g}(0) = (1, 1) > \boldsymbol{0}$ violates both constraints at $t = 0$ by a margin of 1.0 – a direct computational test of the infeasible-initialization guarantee (Lemma 1/Corollary 3) rather than only feasible starts. The LBFN model restores feasibility for this infeasible case within 0.05 s ($\boldsymbol{g}(0.05) = (-1.76, -0.37)$), and the worst-case gradient-error norm across all ten starting points falls to 4.5×10^{-2} by $t = 0.3$ s and to 4.7×10^{-7} by $t = 1$ s. As in Figure 1, a brief transient rise occurs around $t \approx 2$ s as the moving constraint boundary sweeps past several trajectories simultaneously (worst-case gradient-error norm 1.3, worst-case constraint value 2×10^{-4}, i.e., an essentially negligible boundary touch), after which convergence resumes and

the worst-case gradient-error norm falls to 3.1×10^{-9} by $t = 3$ s – a reduction of 8.9 to 9.7 orders of magnitude from each trajectory's initial value – establishing the approach's robustness against both initialization variance and initial infeasibility, a desirable property for computing systems where the optimizer may be restarted or hot-swapped at an arbitrary, possibly infeasible, state.

Comparative analysis against established solvers revealed the LBFN model's superior performance in dynamic programming problems, as shown in Figure 3. The comparative approaches include the active-set (AS) approach [19], the continuous gradient descent (CGD) approach [35], the SQP approach [20], the SeDuMi interior-point approach [36], and the Newton flows (NF) approach [37]. Specifically, the SeDuMi and NF approaches show the largest constraint violations, peaking at 6.7×10^{-1} and 4.7×10^{-1} respectively, during two transition windows ($t \in [1.6, 2.1]$ s and $t \in [7.8, 8.4]$ s); the AS and CGD approaches show smaller, single-window violations (peaking at 3.7×10^{-3} and 1.1×10^{-1} respectively, near $t \approx 1.6$–1.8 s), and the CGD approach additionally develops a persistent steady-state offset rather than decaying (median gradient-error norm 2.5×10^{-1} over $t \in [2, 12]$ s); the LBFN model's own trajectory only briefly touches the constraint boundary during the same transition windows, with a peak excursion of 5.3×10^{-3} – two orders of magnitude smaller than SeDuMi/NF and consistent with the numerical precision of the barrier evaluation (Section 3.1) rather than a genuine, sustained loss of feasibility. Gradient-error convergence metrics confirm distinct advantages: by $t = 12$ s the LBFN model's gradient-error norm has decayed to 1.4×10^{-9} (median 6.7×10^{-9} over $t \in [2, 12]$ s), while the AS, SeDuMi, CGD, and NF baselines remain at 1.3×10^{-1}, 1.3×10^{-1}, 3.5×10^{-1}, and 2.2×10^{-1} respectively at the same instant – four to five orders of magnitude higher. These results confirm the theoretical predictions of global convergence with tunable rates and persistent feasibility.

4.2 Application to Dynamic Robot Navigation

In this part, to further verify the effectiveness of the constructed LBFN model as a real-time computing primitive, a dynamic robot navigation task is designed, in which the robot is required to track a dynamic target in real time. The task is formulated as an inequality-constrained dynamic program with a non-quadratic (Gaussian repulsive potential) objective, to which the general LBFN dynamics of Section 2 apply directly:

$$\begin{aligned} \underset{\boldsymbol{p}(t)\in\mathbb{R}^2}{\text{minimize}} \quad & \frac{1}{2}k_t\|\boldsymbol{p}(t) - \boldsymbol{p}_{Tar}(t)\|^2 + \sum_{i=1}^{7} \exp\left(-\frac{\|\boldsymbol{p}(t) - \boldsymbol{p}_{Obs_i}(t)\|^2}{k_i}\right), \\ \text{subject to} \quad & \mathcal{A}\boldsymbol{p}(t) \leq \boldsymbol{d} \end{aligned} \tag{22}$$

where $\boldsymbol{p}(t) \in \mathbb{R}^2$ denotes the robot's planar position (a specific instance of the decision variable $\boldsymbol{v}(t)$ in (1) for $n = 2$), $\mathcal{A} = [1\ 0; 0\ 1; -1\ 0; 0\ -1]$ and $\boldsymbol{d} = [100; 100; 0; 0]$, so that this inequality denotes the physical bounds of the navigation map. The LBFN model's convergence parameter α is set to 1000, guaranteeing swift convergence and meeting the real-time control requirement of the task (22). The moving target's trajectory is $\boldsymbol{p}_{Tar}(t) = [50{+}40\sin(0.25t),\ 50{+}25\cos(0.25t)]^\top$. The 7 obstacles $\boldsymbol{p}_{Obs_i}(t)$, $i = 1, \ldots, 7$,

are located at $[20; 25]$ m, $[60; 20]$ m, $[70; 60]$ m, $[50; 85]$ m, $[30; 72]$ m, $[38; 35]$ m and $[90; 40]$ m, with parameters $k_1 = 0.8$, $k_2 = 0.6$, $k_3 = 1.0$, $k_4 = 1.2$, $k_5 = 1.2$, $k_6 = 0.8$, $k_7 = 0.9$, and $k_t = 0.3$. The task duration is $T = 30$ s.

As depicted in Figure 4, snapshots at $t = 10$ s, $t = 20$ s, and $t = 30$ s illustrate the experimental results. The dynamic target moves along an elliptical trajectory, with seven stationary obstacles in the map. Figures 4(a), (d), and (g) show the trajectory-tracking result, where the robot starts from its initial point (initially 76.3 m from the moving target) and swiftly converges to the moving target, closing to within 0.01 m by $t = 1$ s and effectively 0 m by $t = 5$ s, and then follows the target in real time while avoiding obstacles (the closest robot-obstacle approach over the entire 30 s run is 1.53 m, near obstacle 5 at $t \approx 23$ s, and the robot remains within the map's $[0, 100] \times [0, 100]$ bounds throughout). Figures 4(b), (e), and (h) show the potential force field on a contour plot, and Figures 4(c), (f), and (i) show the potential-function landscape, for the three time instants. This demonstrates that, under the LBFN model, the robot's onboard computation excellently accomplishes the given dynamic navigation task in real time.

5 Limitations and Future Work

While the LBFN model demonstrates strong theoretical guarantees and empirical performance, several limitations delineate directions for future research relevant to its deployment as a computing primitive. First, although Section 2 shows that Lemma 1, Theorem 2, and Corollary 3 extend verbatim from the quadratic case (1) to any objective that is m_f-strongly convex in $\boldsymbol{v}(t)$ for every t, this still excludes objectives that are only *locally* convex, such as the Gaussian repulsive potential used in Section 4.2, whose Hessian can lose positive definiteness near an obstacle; extending the convergence guarantees to such locally nonconvex, or more generally nonconvex, dynamic programs would broaden applicability but requires new arguments beyond the strong-convexity-based Lyapunov analysis of Section 3. Second, the model assumes that $\mathcal{M}(t)$, $\boldsymbol{g}(t)$, $\mathcal{N}(t)$, $\boldsymbol{d}(t)$ and their time derivatives are known in closed form; in practice these quantities are often estimated from noisy sensor data, so coupling the LBFN dynamics with derivative estimators or observer-based prediction would be necessary for deployment on physical hardware. Third, although Section 3.1 shows the per-instant cost scales as $O(n^3 + pn^2)$, this remains cubic in the state dimension n; for large-scale problems, exploiting the Sherman–Morrison–Woodbury identity for the rank-p Hessian update, warm-started or incremental Cholesky factorizations across nearby time instants, or distributed/parallel implementations across constraint blocks are promising ways to reduce this cost further on multi-core or edge-computing hardware. Fourth, the barrier parameter $w(t) = w(0)e^{\gamma_w t}$ grows without bound, which is necessary for asymptotic exactness but can induce numerical ill-conditioning of $\nabla_{\boldsymbol{vv}}\Upsilon$ near the constraint boundary over long horizons; adaptive or saturating barrier schedules merit investigation for long-duration deployments. Finally, the robot navigation case study in Section 4.2 is validated in simulation; hardware-in-the-loop and physical robot experiments, together with benchmarking on higher-dimensional

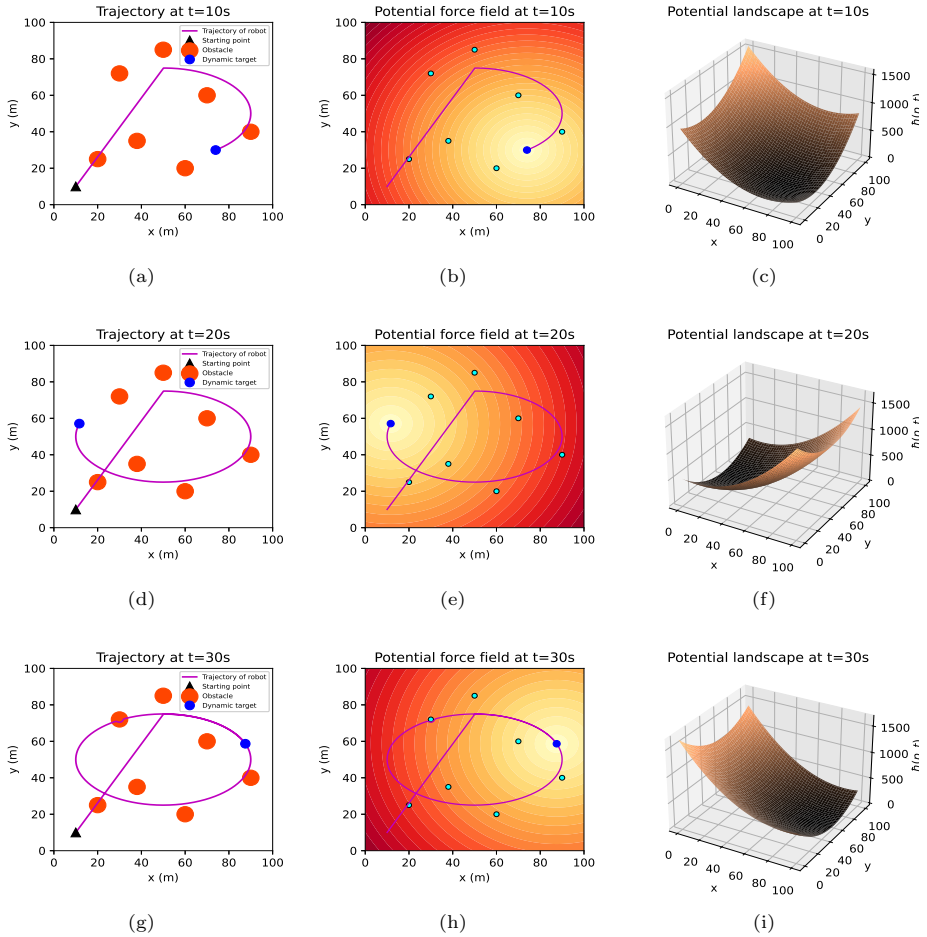

Fig. 4 Experimental results of the dynamic robot navigation task (22) accomplished by the LBFN model in real time, task duration $T = 30$ s, involving 7 obstacles. (a) Trajectory at $t = 10$ s. (b) Potential force field at $t = 10$ s. (c) Potential landscape at $t = 10$ s. (d)–(f) Same at $t = 20$ s. (g)–(i) Same at $t = 30$ s.

and multi-agent DQP-IC instances, are natural next steps to further substantiate the real-time computing applicability of the proposed model.

6 Conclusion

This paper has established a logarithmic barrier function-based Newton's (LBFN) computational model for real-time solving of dynamic quadratic programming problems subject to inequality constraints. The proposed model fundamentally advances beyond traditional approaches by employing parametric adaptive log-barrier functions to directly handle inequality constraints, thereby avoiding the auxiliary variables typically required by KKT-based methods. We established the well-posedness of the

resulting continuous-time dynamics, proved global asymptotic stability with an exponentially decaying, tunable tracking error via a Lyapunov argument, and derived a per-instant computational complexity of $O(n^3 + pn^2)$ comparable to a single Newton step on a static quadratic program of the same size. These properties make the model particularly suitable for real-time computing environments where both computational efficiency and constraint adherence are critical. Numerical experiments comprehensively validated the model's performance, showing superior convergence speed and solution accuracy compared to active-set, sequential quadratic programming, interior-point, continuous gradient-descent, and Newton-flow baselines, while a dynamic robot navigation case study demonstrated simultaneous trajectory tracking and obstacle avoidance in real-time operation. Building on the limitations discussed in Section 5, future work will extend the LBFN framework to nonlinear and nonconvex dynamic programs, incorporate noise-robust parameter estimation, exploit factorization-update techniques for large-scale and distributed computing deployments, and pursue hardware validation on physical robotic and embedded computing platforms.

Acknowledgments. Not applicable.

Declarations

- **Funding:** This research received no external funding.
- **Conflict of interest/Competing interests:** The authors declare no competing interests.
- **Ethics approval:** Not applicable. This study did not involve human participants, human data, or animals.
- **Consent to participate:** Not applicable.
- **Consent for publication:** Not applicable.
- **Availability of data and materials:** The data generated and analysed during the current study, together with the scripts that reproduce the numerical experiments and figures, are available in the GitHub repository at https://github.com/yzhou364/LBFN.
- **Code availability:** The Python implementation of the LBFN model and all baseline solvers is available in the GitHub repository at https://github.com/yzhou364/LBFN.
- **Authors' contributions:** Y.Z. conceived the research idea, supervised the study, and revised the manuscript; Y.Z. is the corresponding author. J.W. developed the theoretical framework, implemented the LBFN algorithm, conducted the numerical simulations and the robot navigation experiment, and drafted the manuscript. Both authors reviewed and approved the final version of the manuscript.
- **Use of AI:** [DRAFT – authors: replace with an accurate account before submission, per the journal's AI-use policy.] An AI assistant (Claude, Anthropic) was used to implement the numerical simulation code, execute the numerical experiments reported in Section 4, and assist with drafting and revising the manuscript text. The authors designed the study, verified the correctness of the implementation and results, and take full responsibility for the content of this manuscript.

References

[1] D. Liu, S. Xue, B. Zhao, B. Luo and Q. Wei, "Adaptive Dynamic Programming for Control: A Survey and Recent Advances," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 51, no. 1, pp. 142–160, Jan. 2021.

[2] Y. E. Wang, "Theory of Broadband Transmission of Frequency Modulated Signals through High-Q Resonators with Continuous Time-Varying Matching Networks," *IEEE Access*, doi: 10.1109/ACCESS.2025.3595056.

[3] D. Hanover, A. Loquercio, L. Bauersfeld, A. Romero, R. Penicka, Y. Song, G. Cioffi, E. Kaufmann and D. Scaramuzza, "Autonomous Drone Racing: A Survey," *IEEE Transactions on Robotics*, vol. 40, pp. 3044–3067, May 2024.

[4] J. Park, M. Kang, Y. Lee, J. Jung, H. -T. Choi and J. Choi, "Multiple Autonomous Surface Vehicles for Autonomous Cooperative Navigation Tasks in a Marine Environment: Development and Preliminary Field Tests," *IEEE Access*, vol. 11, pp. 36203–36217, 2023.

[5] K. Harlow, H. Jang, T. D. Barfoot, A. Kim and C. Heckman, "A New Wave in Robotics: Survey on Recent MmWave Radar Applications in Robotics," *IEEE Transactions on Robotics*, vol. 40, pp. 4544–4560, Sept. 2024.

[6] Y. Liufu et al., "Collaborative Physics-Informed Neural Dynamics Approach for Autonomous Vehicles with Nonconvex Safety-Critical Constraints", *IEEE Transactions on Intelligent Transportation Systems*, doi: 10.1109/TITS.2025.3572336.

[7] J. Nur Fadila, N. Haliza Abdul Wahab, A. Alshammari, A. Aqarni, A. Al-Dhaqm and N. Aziz, "Comprehensive Review of Smart Urban Traffic Management in the Context of the Fourth Industrial Revolution," *IEEE Access*, vol. 12, pp. 196866–196886, 2024.

[8] C. Creβ, Z. Bing and A. C. Knoll, "Intelligent Transportation Systems Using Roadside Infrastructure: A Literature Survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 7, pp. 6309–6327, Jul. 2024.

[9] M. N. Elsayed, O. De Silva, A. Jayasiri, G. K. I. Mann and R. Gosine, "Optimization-Based Maneuvering Target Tracking Using Multiple Model Horizon Scenario Tree With Model Interaction," *IEEE Transactions on Aerospace and Electronic Systems*, doi: 10.1109/TAES.2025.3592177.

[10] G. R. Prudhvi Kumar and S. Dasarathan, "Metaheuristic-Tuned Droop Control for PV-Based DC Microgrid Optimization Under Dynamic Load Conditions," *IEEE Access*, doi: 10.1109/ACCESS.2025.3592751.

[11] J. Fan, L. Jin, P. Li, J. Liu, Z.-G. Wu and W. Chen, "Coevolutionary Neural Dynamics Considering Multiple Strategies for Nonconvex Optimization,"

Tsinghua Science and Technology, doi: 10.26599/TST.2025.9010120.

[12] Y. Liufu et al., "Modified Gradient Projection Neural Network for Multiset Constrained Optimization," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 9, pp. 9413–9423, Sept. 2023.

[13] J. M. Altschuler, J. Bok and K. Talwar, "On the Privacy of Noisy Stochastic Gradient Descent for Convex Optimization," *SIAM Journal on Computing*, vol. 53, no. 4, 969–1001, 2024.

[14] H. Wen and X. He, "Arbitrary-Time Stable Gradient Flows Using Neurodynamic System for Continuous-Time Optimization," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 12, pp. 4559–4563, Dec. 2023.

[15] R. Chen and X. Li, "Implicit Runge-Kutta Methods for Accelerated Unconstrained Convex Optimization," *IEEE Access*, vol. 8, pp. 28624–28634, 2020.

[16] Jing Ren, K. A. McIsaac and R. V. Patel, "Modified Newton's Method Applied to Potential Field-Based Navigation for Mobile Robots," *IEEE Transactions on Robotics*, vol. 22, no. 2, pp. 384–391, Apr. 2006.

[17] D. Niu, Y. Hong and E. Song, "A Dual Inexact Nonsmooth Newton Method for Distributed Optimization," *IEEE Transactions on Signal Processing*, vol. 73, pp. 188–203, 2025.

[18] H. Kim and H. Park, "Nonnegative Matrix Factorization Based on Alternating Nonnegativity Constrained Least Squares and Active Set Method," *SIAM Journal on Matrix Analysis And Applications*, vol. 30, no. 2, pp. 713–730. 2008.

[19] G. Cimini and A. Bemporad, "Exact Complexity Certification of Active-Set Methods for Quadratic Programming," *IEEE Transactions on Automatic Control*, vol. 62, no. 12, pp. 6094–6109, Dec. 2017.

[20] F. E. Curtis and M. L. Overton, "A Sequential Quadratic Programming Algorithm for Nonconvex, Nonsmooth Constrained Optimization," *SIAM Journal on Optimization*, vol. 22, no. 2, pp. 474–500. 2012.

[21] A. S. Berahas, M. Xie and B. Zhou, "A Sequential Quadratic Programming Method With High-Probability Complexity Bounds for Nonlinear Equality-Constrained Stochastic Optimization," *SIAM Journal on Optimization*, no. 35, no. 1, pp. 240–269, 2025.

[22] X. Shi, X. Xu, J. Cao, and X. Yu, "Finite-Time Convergent Primal-Dual Gradient Dynamics With Applications to Distributed Optimization," *IEEE Transactions on Cybernetics*, vol. 53, no. 5, pp. 3240–3252, May 2023.

[23] J. I. Poveda and M. Krstic, "Fixed-time Gradient-Based Extremum Seeking," in Proceedings of the American Control Conference, 2020, pp. 2838–2843.

[24] I. Fatkhullin and B. Polyak, "Optimizing Static Linear Feedback: Gradient Method," *SIAM Journal on Control and Optimization*, vol. 59, no. 5, pp. 3887–3911, 2021.

[25] M. Mestari, M. Benzirar, N. Saber and M. Khouil, "Solving Nonlinear Equality Constrained Multiobjective Optimization Problems Using Neural Networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 26, no. 10, pp. 2500–2520, Oct. 2015.

[26] T. Breiten and L. Pfeiffer, "On the Turnpike Property and The Receding-Horizon Method for Linear-Quadratic Optimal Control Problems," *SIAM Journal on Control and Optimization*, vol. 58, no. 2, pp. 1077–1102, 2020.

[27] Y. Hu, L. Wei, X. Guo, Z. Li, Y. Wei, and Q. Fang, "Priority-Based Reward Mechanism for Dual-Robot Path Planning in Dynamic Environments," *IEEE Access*, vol. 13, pp. 92519-92530, 2025.

[28] M. Liu, Y. Li, Y. Chen, Y. Qi, and L. Jin, "A Distributed Competitive and Collaborative Coordination for Multirobot Systems," *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 11436-11448, Dec. 2024.

[29] R. J. Roesthuis and S. Misra, "Steering of Multisegment Continuum Manipulators Using Rigid-Link Modeling and FBG-Based Shape Sensing," *IEEE Transactions on Robotics*, vol. 32, no. 2, pp. 372–382, Apr. 2016.

[30] J. Lin, D.-H. Zhai, Y. Xiong and Y. Xia, "Safety Control for UR-Type Robotic Manipulators via High-Order Control Barrier Functions and Analytical Inverse Kinematics," *IEEE Transactions on Industrial Electronics*, vol. 71, no. 6, pp. 6150–6160, Jun. 2024.

[31] S. Jing, Y. Zhao, X. Zhao, F. Hui and A. J. Khattak, "An Efficient High-Risk Lane-Changing Scenario Edge Cases Generation Method for Autonomous Vehicle Safety Testing," *IEEE Transactions on Intelligent Vehicles*, vol. 10, no. 1, pp. 459–471, Jan. 2025.

[32] L. Jin, L. Wei, and S. Li, "Gradient-Based Differential Neural-Solution to Time-Dependent Nonlinear Optimization," *IEEE Transactions on Automatic Control*, vol. 68, no. 1, pp. 620–627, Jan. 2023.

[33] Y. Liufu et al., "Adaptive Noise-Learning Differential Neural Solution for Time-Dependent Equality-Constrained Quadratic Optimization," *IEEE Transactions on Neural Networks and Learning Systems*, doi: 10.1109/TNNLS.2025.3561415.

[34] S. Boyd and L. Vandenberghe, Convex Optimization. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[35] J. Sirignano and K. Spiliopoulos, "Stochastic Gradient Descent in Continuous Time," *SIAM Journal on Financial Mathematics*, vol. 8, no. 1, pp. 933–961, 2017.

[36] Y. Labit, D. Peaucelle and D. Henrion, "SeDuMi Interface 1.02: A tool for Solving LMI Problems with SeDuMi," in Proceedings. IEEE International Symposium on Computer Aided Control System Design. IEEE, 2002: 272–277.

[37] D. Niu, Y. Hong and E. Song, "A Dual Inexact Nonsmooth Newton Method for Distributed Optimization," *IEEE Transactions on Signal Processing*, vol. 73, pp. 188–203, 2025.